# P2197R0
# Formatting for std::complex

Working Draft, 2019-12-22

**This version:**
http://fmt.dev/papers/p2197r0.html

**Authors:**
Michael Tesch
Victor Zverovich

**Audience:**
LEWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

This paper discusses extending coverage of the formatting functionality of [P0645] to `std::complex`.

# Table of Contents



## § 1. Introduction

[P0645] has proposed a text formatting facility that provides a safe and extensible alternative to the `printf` family of functions. This paper explores how to format complex numbers represented by `std::complex`.

## § 2. Examples

Default format:

```
std::string s = std::format("{}", 1.0 + 2i); // s == "(1+2i)"
```

iostreams-compatible format (optional):

```
std::string s = std::format("{:p}", 1.0 + 2i); // s == "(1,2)"
```

Format specifiers:

```
std::string s = std::format("{:.2f}", 1.0 + 2i); // s == "1.00+2.00i"
```

## § 3. Motivation

This is a proposal defining formatting of complex numbers represented by the library type `std::complex`. The default notation `(3+4i)` is proposed, as it is common in mathematics, the physical sciences, and many other popular mathematical software environments. This form is also more consistent with the standard library literals for `std::complex` [N3660]. In addition to defining the new format and discussing design choices, this proposal attempts to address questions around introducing a format which differs from the existing iostreams format, and why the aforementioned advantages outweigh the disadvantages of introducing a potentially incompatible format. An option to produce iostreams-compatible output is also provided.

The formatting of `std::complex` should be simple, consistent with existing conventions of `<format>`, and conveniently support the most common use cases of `std::complex`. As the first nested format specified for `<format>`, it can also serve as an example for how format nesting can be done.

Mathematics generally follows the convention that complex numbers consist of a real part and an orthogonal imaginary part which is identified by multiplication of the imaginary unit vector . Extending the set of unit vectors in this way furthermore implies straightforward extensions to other useful algebras such as quaternions , dual numbers , etc.

For the types `std::complex<{float,double,long double}>`, C++14 introduced string literals to the standard library in the namespace `std::complex_literals`. These string literals

acknowledge the common use cases of these types and provide a convenient way to write complex numbers in code, for example the number can be written in code as as `1.0f + 1if`, `1.0 + 1i`, or `1.0l + 1il`, depending on the desired underlying type.

Sometimes it is possible to omit one part in a symbolic representation yet retain bijectivity in the machine representation to symbolic mapping. For example, the complex number can be unambiguously written as either `0` or `0i`. The convention of mathematics is the former, although the latter has the advantage of implying the underlying field.

As specified in [N4849], the existing iostreams formatting of a complex number `x` is essentially

```
s << '(' << x.real() << "," << x.imag() << ')';
```

where `s` is a stream object.

This embedded comma can cause silent unexpected generation of ambiguous output, which can happen e.g. when the locale's decimal separator is set to comma. This ambiguity does not exist in the imaginary unit notation, even when an unusual locale is used.

## § 4. Design Considerations

With an eye to providing a replacement for all the functionality of iostreams, the following considerations are made.

### § 4.1. Numeric form

The question of how to represent the numeric type `T` of `std::complex<T>` is simply delegated to the `formatter<T>` for that type. Special alignment, fill, and sign rules may apply when `T` is `float`, `double` or `long double`, but other custom value types are accomodated. This is done by optionally forwarding a designated portion of the `formatter<std::complex<T>>` format spec to `formatter<T>`.

Although the standard does not specify behavior of `std::complex<T>` for types other than `float`, `double`, `long double`, it is not uncommon to use a type for `T` which provides functionality such as extended precision or automatic differentiation. The formatting specification should therefore be recursive, so that arbitrary numerical types for `T` are properly formatted.

## § 4.2. Imaginary unit

As previously mentioned, mathematics notation typically uses  as the complex unit vector, however it is very common in electrical engineering to use  instead. Mathematica uses the Unicode character *ⅈ* for the imaginary unit. Another common written form of complex numbers puts the imaginary unit in front of the imaginary part rather than after it. Julia uses the dual-character symbol `im`, and it it easy to imagine wanting to explicitly specify the usually-omitted implied real unit-vector, result in a format like `3re + 4im`. Supporting these use cases would be nice, but not with significant implementation difficulty.

## § 4.3. Omission of a part

Because the complex number is always a pair of real part and imaginary part, it is not necessary to print both parts if one of the parts is identical to a known quantity: typically (nonnegative) zero; in this case omission implies the value uniquely. Either the real or the imaginary part can be omitted when this condition is satisfied, although clearly not both.

Should a part be dropped?

The benefits of part dropping include: shorter conversions in the special but common cases of purely real or imaginary numbers, adherence to common notation. There is also a tie-in with the design consideration discussed below of whether surrounding parenthesis are necessary: a single numeric value does not need to be surrounded by parenthesis in order to recognize it as the value for an entire complex number.

What are the conditions under which a part can be dropped?

A simple comparison with zero is usually insufficient to decide whether a part can be omitted. While C++ does not specify the underlying floating-point format, for correct round-trip conversions, the omitted part must be binary equivalent to `T(0)`. The function `std::signbit<T>` is used to distinguish between `-0` and `0`, so the type `T` must have both a defined `std::formatter<T>` and `std::signbit<T>` to distinguish the two cases.

This nuance is demonstrated by the result of `sqrt(-1. + 0i)` vs `sqrt(-1. - 0i)`.

Which part should be dropped?

Either part of an imaginary number could be dropped if it is binary equal to `T(0)`, but in the special case of  dropping both parts would lead to the absurdity of an empty string. This is an open question, but it is the opinion of the author that the real part should be dropped, so that the remaining symbolic representation retains the imaginary unit vector, indicating use of the complex field .

## § 5. Parentheses

Should parentheses be mandatory?

Are parentheses always neccesary to unambiguously specify a complex number?

Do mandatory parentheses significantly improve ease or speed of complex number parsing?

If parentheses are not mandatory, when should they be omitted?

## § 6. Backwards Compatibility

To maintain backward compatibility we propose an easy-to-use format specifier that exactly reproduces the legacy iostreams output format.

The `ios` specifiers that affect complex number output are `precision` and `width`, these can not be easily guessed, but can be specified manually in the nested format specifier. Otherwise the compatibilty format the output will produce roughly the same output (modulo locale and default format for `formatter<T>`) that iostreams produces.

## § 7. Parsing

This paper does not address parsing (scan'ing) for the type `std::complex<T>` but does aim to produce formatted output that can unambiguously round trip formatted and parsed.

## § 8. Survey of other languages

The following programming languages/environments similarly use the imaginary-unit notation as their default: Python, Julia, R, MATLAB, Mathematica, Go. If you know the type of the data, these languages offer round-trip conversion from complex -> text -> complex, but because some of them drop the complex part in their textual output when the complex part is zero (or even negative zero!) some arguably pertinent information can be lost during formatting.

| Language | Basic Format | Result of `sqrt(-1)` | Result of `sqrt(-1) - sqrt(-1)` |
|---|---|---|---|
| C++ iostreams | `(3,4)` | `(0,1)` | `(0,0)` |
| NumPy | `(3+4j)` | `1j` | `0j` |
| Julia | `3.0 + 4.0im` | `0.0 + 1.0im` | `0.0 + 0.0im` |
| Octave | `3 + 4i` | `0 + 1i` | `0` |

| Language | Basic Format | Result of `sqrt(-1)` | Result of `sqrt(-1) - sqrt(-1)` |
|---|---|---|---|
| Mathematica* | `1+i` | `i` | `0` |
| R | `(3+4i)` | `(0+1i)` | `(0+0i)` |
| C++14 literals | `3.0 + 4i` | `1i` | `0i` |
| Go | `(3+4i)` | `(0+1i)` | `(0+0i)` |

`*` - checked via wolframalpha

Haskell provides `a :+ b` notation - this choice does not need much commentary, this much is offered: it is quite unique.

C# does not provide this functionality, but the doc page for complex includes an example code for creating an appropriate formatter.

## § 9. Wish List

Feature wish list:

- nested specification of real and imaginary parts via `formatter<T>`
- easy substitution of "old style" iostreams format with simply `{:p}`
- defineable symbol for imaginary unit (`j`, `im`)
- option to prefix the imaginary part with the imaginary unit
- control over which (real/imag) part omission (`0` or `0j`)
- default to minimalist unique parseable format: `1`, `1i`, `0j`, `(1+1i)`
- toggle to turn off surrounding parens: `1+1i`
- toggle to turn off outputting `-` on negative zero - addressed by P2021
- center alignment `^` aligns output around the connecting `+/-`
- option for polar formatted output, ie `(1.41421*exp(i*3.14159))`

## § 10. Proposed Wording

Modify [complex.syn] as follows:

```
template<class T, class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, cor

// 26.4.?, formatting
template<class charT> struct formatter<complex<float>, charT>;
template<class charT> struct formatter<complex<double>, charT>;
template<class charT> struct formatter<complex<long double>, charT>;
```

Add a new section 26.4.? Formatting [complex.format]:

Each `formatter<complex<T>, charT>` (format.formatter) specialization in this section meets the *Formatter* requirements (formatter.requirements). The `parse` member functions of these formatters interpret the format specification as *std-format-spec* (format.string.std) except that the `0` option is invalid.

```
template<class charT> struct formatter<complex<T>, charT> {
  typename basic_format_parse_context<charT>::iterator
    parse(basic_format_parse_context<charT>& ctx);

  template<class FormatContext>
    typename FormatContext::iterator
      format(const complex<T>& c, FormatContext& ctx);
};

template<class FormatContext>
  typename FormatContext::iterator
    format(const complex<T>& c, FormatContext& ctx);
```

Let `real = format(ctx.locale(), "{:<format-specs>}", c.real())` and `imag = format(ctx.locale(), "{:<format-specs>}", c.imag())`, where `<format-specs>` is *std-format-spec* with *fill-and-align* and *width* removed.

*Effects:* Equivalent to:

```
 format_to(ctx.out(), "{:<fill-align-width>}",
           format(c.real() != 0 ? "({0}+{1}i)" : "{1}i", real, imag))
```

where `<fill-align-width>` is the *fill-and-align* and *width* part of *std-format-spec*. If alignment is not specified `>` is used.

## § 11. Questions

**Q1**: Do we want any of this?

**Q2**: The strategy of this paper is to include a laundry list of possibilities, which parts do we want?

## § References

### § Informative References


**[N3660]**

Peter Sommerlad. User-defined Literals for std::complex, part 2 of UDL for Standard Library Types (version 4). 19 April 2013. URL: https://wg21.link/n3660

**[N4849]**

Richard Smith. Working Draft, Standard for Programming Language C++. URL: https://wg21.link/n4849

**[P0645]**

Victor Zverovich. Text Formatting. URL: https://wg21.link/p0645